Traducción Automática de Escenarios BDD en Lenguaje Natural a Lenguaje Intermedio de BDD mediante Técnicas de Procesamiento de Lenguaje Natural

por
Anna Paula Aimeri Vicentin

Presentado ante la Facultad de Matemática, Astronomía, Física y Computación como parte de los requerimientos para la obtención del grado de Licenciado en Ciencias de la Computación de la

Universidad Nacional de Córdoba

MES, 2026

Directora:
Laura Brandán Briones

Agradecimientos

Resumen

Abstract

Índice general

1. Introducción **7**
1.1. Motivación y Contexto 7
1.2. Problema de Investigación 8
1.3. Enfoque Propuesto 9
1.4. Estructura del Documento 9
1.5. Trabajo Realizado 9

2. Marco Teórico **10**
2.1. Desarrollo Dirigido por Comportamiento 10
2.1.1. Sintaxis Gherkin 11
2.2. Pruebas Basadas en Modelos 12
2.3. Sistemas de Transición Simbólicos 13
2.4. Lenguaje Intermedio de BDD 14
2.4.1. Información Específica del Dominio 14
2.4.2. Gramática IBDD 15
2.4.3. Pipeline de Transformación 16
2.5. Modelos de Lenguaje Grandes 17
2.5.1. Aplicaciones en Ingeniería de Software 17
2.5.2. Limitaciones 18
2.6. Trabajos Relacionados 18
2.6.1. Pipeline Formal BDD-a-MBT 18
2.6.2. Automatización de BDD mediante NLP 19
2.6.3. LLMs para la Automatización de BDD 19
2.6.4. Posicionamiento del Presente Trabajo 19
2.7. Análisis Sintáctico y Gramáticas Formales 20
2.7.1. Notación EBNF 20
2.7.2. Algoritmo de Earley 20
2.7.3. Lark como Herramienta de Análisis 21

3. Enfoque y Metodología **22**
3.1. Selección del Modelo de Lenguaje 22
3.1.1. Metodología de Evaluación 22
3.1.2. Resultados de la Evaluación Comparativa 23
3.1.3. Validación con Literatura Existente 25
3.1.4. Decisión Final 26
3.2. Selección de la Herramienta de Análisis Sintáctico 27
3.2.1. Lark: Características y Algoritmos 27
3.2.2. Adopción Industrial y Académica 28
3.2.3. Manejo de Gramáticas Ambiguas 28
3.2.4. Decisión: Earley en Lark 29
3.3. Diseño del Prompt de Traducción 29
3.3.1. Estructura del Prompt 29
3.3.2. Aprendizaje por Ejemplos 30
3.3.3. Reglas de Traducción 31
3.3.4. Formato de Intercambio 31
3.4. Construcción del Dataset 32
3.4.1. Escenarios Canónicos de Referencia 32
3.4.2. Generación Sintética de Escenarios 33

4. Implementación **34**
4.1. Arquitectura del Sistema 34
4.2. Definición de la Gramática IBDD 35
4.2.1. Estructura de la Gramática 35
4.2.2. Jerarquía de Expresiones 36
4.2.3. Soporte de Operadores 37
4.2.4. Transformación del Árbol Sintáctico 37
4.3. Módulo de Traducción 37
4.3.1. Abstracción del Proveedor de LLM 38
4.3.2. Structured Outputs de OpenAI 38
4.3.3. Inferencia Dinámica del Esquema 39
4.3.4. Procesamiento de Escenarios 40
4.4. Módulo de Validación Sintáctica 41
4.5. Módulo de Corrección Iterativa 41
4.5.1. Identificación de Casos Fallidos 42
4.5.2. Análisis de Errores 42
4.5.3. Reintento con Contexto de Error 43
4.5.4. Fusión No Destructiva y Re-validación 43

5. Evaluación **45**
5.1. Preguntas de Investigación 45
5.2. Diseño Experimental 45
5.3. Dataset 45
5.4. Métricas 45
5.5. Resultados 45
5.6. Análisis 45

6. Discusión **46**
6.1. Interpretación de Resultados 46
6.2. Limitaciones 46
6.3. Amenazas a la Validez 46

7. Conclusiones y Trabajo Futuro **47**
7.1. Trabajo Futuro 47

Capítulo 1

Introducción

1.1. Motivación y Contexto

El desarrollo de software moderno enfrenta un desafío persistente: cerrar la brecha entre la especificación del comportamiento deseado de un sistema y la generación automática de casos de prueba que validen ese comportamiento. El Desarrollo Dirigido por Comportamiento (BDD, *Behavior-Driven Development*) ha demostrado ser efectivo para capturar requisitos mediante escenarios en lenguaje natural estructurado, utilizando la sintaxis Given-When-Then del formato Gherkin. Esta aproximación facilita la colaboración entre técnicos y no técnicos, permitiendo que analistas de negocio, testers y desarrolladores compartan un lenguaje común para describir el comportamiento esperado del sistema.

Por otro lado, las Pruebas Basadas en Modelos (MBT, *Model-Based Testing*) ofrecen un enfoque riguroso y automatizado para generar casos de prueba a partir de modelos formales del sistema, como los Sistemas de Transición Simbólicos (STS, *Symbolic Transition Systems*). Estos modelos formales permiten aplicar algoritmos de generación automática de tests que garantizan cobertura exhaustiva y detectan comportamientos inesperados.

Sin embargo, existe una discontinuidad entre estos dos mundos. Los escenarios BDD, expresados en lenguaje natural, carecen de la precisión formal necesaria para alimentar directamente las herramientas de MBT. Investigaciones recientes han propuesto el Lenguaje Intermedio de BDD (IBDD) como puente entre ambos paradigmas [30]. IBDD mantiene la estructura Given-When-Then de BDD pero incorpora elementos formales de los STS, incluyendo variables globales y locales, guardas lógicas, gates de interacción y asignaciones explícitas de estado.

Los trabajos de Zameni et al. han demostrado que, una vez que los escena-

rios están representados en IBDD, es posible transformarlos automáticamente en Sistemas de Transición BDD (BDDTS) y posteriormente en STS, desde donde se pueden generar casos de prueba completos y formalmente válidos [31]. Sin embargo, un eslabón de esta cadena de automatización aún no ha sido abordado: la traducción inicial de escenarios Gherkin escritos por humanos a representaciones IBDD formalmente correctas. La Figura 1.1 ilustra este pipeline y señala la etapa que aborda el presente trabajo.

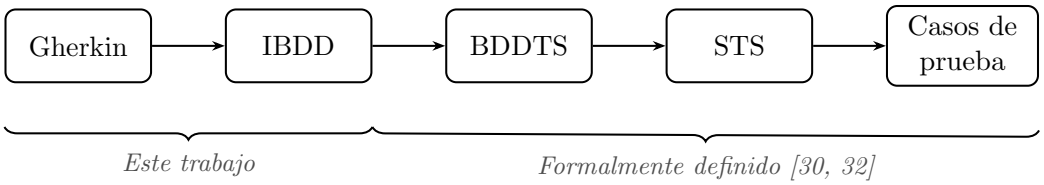

Figura 1.1: Pipeline de transformación BDD-a-MBT y ubicación del presente trabajo

Actualmente, esta traducción es un proceso enteramente manual que requiere experiencia especializada en teoría de sistemas de transición y conocimiento profundo de la gramática IBDD. Un analista debe identificar variables implícitas, inferir gates de interacción apropiados, establecer guardas lógicas correctas y determinar las asignaciones de estado necesarias—información que típicamente no está explícita en el texto natural del escenario BDD. Este cuello de botella limita la escalabilidad industrial de todo el pipeline BDD-a-MBT.

1.2. Problema de Investigación

El problema central que aborda este trabajo es: *¿cómo automatizar la traducción de escenarios BDD en formato Gherkin al lenguaje intermedio formal IBDD, de manera que se preserve la semántica del comportamiento especificado y se generen representaciones formalmente válidas para alimentar herramientas de generación automática de casos de prueba?*

Este problema presenta múltiples desafíos técnicos. En primer lugar, los escenarios Gherkin contienen imprecisiones inherentes al lenguaje natural que deben resolverse para producir especificaciones formales. A esto se suma que elementos cruciales de IBDD—variables globales, tipos de datos, funciones del dominio, gates del sistema—no están explícitos en el texto Gherkin y deben inferirse del contexto o del conocimiento del dominio. La correspondencia entre construcciones lingüísticas en Gherkin y elementos formales de IBDD tampoco es trivial; por ejemplo, la frase “el usuario hace clic en agregar al

carrito" debe traducirse a un gate de entrada específico (`?click.add_button`) con variables de interacción apropiadas. Las traducciones generadas, a su vez, deben ser sintácticamente correctas según la gramática IBDD y semánticamente consistentes con la especificación BDD original. Finalmente, la diversidad de dominios que abarcan los escenarios BDD—e-commerce, sistemas industriales, aplicaciones financieras—introduce vocabularios y patrones de interacción específicos que el sistema debe manejar.

1.3. Enfoque Propuesto

Este trabajo propone un enfoque híbrido que combina las capacidades de comprensión semántica de Modelos de Lenguaje Grandes (LLMs, *Large Language Models*) con la estructura formal proporcionada por la gramática IBDD y técnicas de análisis sintáctico determinístico.

La arquitectura del sistema se organiza en torno a dos componentes principales. El primero es un módulo de traducción basado en LLM que utiliza modelos de lenguaje pre-entrenados para interpretar escenarios Gherkin y generar candidatos de traducción IBDD, aprovechando la capacidad de los LLMs para capturar patrones semánticos complejos y realizar inferencias sobre información implícita. El segundo componente es un módulo de validación sintáctica que emplea la biblioteca Lark con su analizador Earley para verificar las traducciones generadas contra la gramática formal IBDD, garantizando corrección estructural.

El flujo del sistema opera de la siguiente manera: un escenario Gherkin es procesado por el LLM, que genera tres candidatos de traducción IBDD; estos candidatos son luego validados sintácticamente mediante Lark. Si la validación falla, se inicia un ciclo de corrección iterativa hasta obtener una representación IBDD válida.

La elección de LLMs se fundamenta en su demostrada capacidad para tareas de traducción entre lenguajes formales y naturales, como se evidencia en trabajos recientes sobre generación de código desde especificaciones [12] y traducción de requisitos a modelos formales [23]. Sin embargo, los LLMs por sí solos no garantizan corrección sintáctica ni consistencia formal, de ahí la necesidad del componente de validación.

1.4. Estructura del Documento

1.5. Trabajo Realizado

Capítulo 2

Marco Teórico

Este capítulo presenta los conceptos fundamentales necesarios para comprender el desarrollo del presente trabajo. Se introducen el Desarrollo Dirigido por Comportamiento y su sintaxis Gherkin, las Pruebas Basadas en Modelos y los Sistemas de Transición Simbólicos que las sustentan, el Lenguaje Intermedio de BDD que actúa como puente entre ambos paradigmas, los Modelos de Lenguaje Grandes y las técnicas de análisis sintáctico basadas en gramáticas formales. Finalmente, se revisan los trabajos relacionados que contextualizan esta investigación.

2.1. Desarrollo Dirigido por Comportamiento

El Desarrollo Dirigido por Comportamiento (BDD, *Behavior-Driven Development*) es una práctica de desarrollo de software que surge como evolución del Desarrollo Dirigido por Pruebas (TDD, *Test-Driven Development*). Fue propuesto por Dan North en 2006 [17] como respuesta a las dificultades que encontraban los equipos al adoptar TDD: incertidumbre sobre qué probar, cómo nombrar las pruebas y cómo entender por qué una prueba falla.

La innovación central de BDD consiste en desplazar el foco desde las pruebas hacia el comportamiento esperado del sistema. En lugar de escribir pruebas unitarias que verifican implementaciones específicas, BDD propone describir el comportamiento del sistema desde la perspectiva del usuario mediante escenarios concretos. Este cambio de vocabulario—de "prueba" a "comportamiento"—tiene implicaciones profundas: facilita la comunicación entre desarrolladores, analistas de negocio y otros *stakeholders*, y produce especificaciones que sirven simultáneamente como documentación y como base para pruebas automatizadas.

Solís y Wang [24] identifican como características fundamentales de BDD la

ubicuidad del lenguaje compartido entre todos los participantes del proyecto, la naturaleza iterativa del proceso de especificación, y el uso de ejemplos concretos como mecanismo primario de comunicación entre el dominio del negocio y el dominio técnico.

2.1.1. Sintaxis Gherkin

El vehículo principal para expresar escenarios BDD es Gherkin, un lenguaje de dominio específico diseñado para ser legible tanto por humanos como por herramientas de automatización [4]. Un escenario Gherkin describe una situación concreta del comportamiento del sistema mediante tres cláusulas estructuradas:

- **Given** (*Dado*): establece las precondiciones o el estado inicial del sistema antes de que ocurra la acción bajo prueba.
- **When** (*Cuando*): describe la acción o evento que desencadena el comportamiento que se desea especificar.
- **Then** (*Entonces*): define el resultado esperado o la postcondición que debe verificarse tras la ejecución de la acción.

A modo de ejemplo, el siguiente escenario especifica el comportamiento de un sistema de comercio electrónico al agregar un producto al carrito de compras:

Listing 2.1: Ejemplo de un escenario BDD en sintaxis Gherkin.

```
Scenario: Add product to shopping cart
  Given the user is on the product page for "Laptop"
  And the product is in stock
  When the user clicks the "Add to Cart" button
  Then the cart should contain 1 item
  And the total price should be updated
```

Los escenarios Gherkin se agrupan en archivos denominados *features*, donde cada *feature* describe una funcionalidad del sistema e incluye uno o más escenarios que ilustran su comportamiento esperado bajo distintas condiciones. Las palabras clave **And** y **But** permiten encadenar múltiples condiciones dentro de una misma cláusula, mejorando la legibilidad sin alterar la semántica de la especificación.

Esta estructura, si bien es lo suficientemente formal para ser procesada por herramientas de automatización como Cucumber[1], mantiene un nivel de abstracción cercano al lenguaje natural. Es precisamente esta característica la que convierte a Gherkin en un formato accesible para *stakeholders* no técnicos, pero también la que introduce desafíos de ambigüedad e imprecisión inherentes al uso del lenguaje natural. Estos desafíos se vuelven críticos cuando se busca integrar los escenarios BDD con técnicas de verificación formal, como las Pruebas Basadas en Modelos que se describen a continuación.

2.2. Pruebas Basadas en Modelos

Las Pruebas Basadas en Modelos (MBT, *Model-Based Testing*) constituyen un enfoque de verificación de software en el que los casos de prueba se derivan de manera rigurosa a partir de un modelo formal que describe el comportamiento esperado del sistema bajo prueba [26]. A diferencia de las técnicas de prueba manuales, donde un ingeniero diseña casos de prueba individualmente, MBT automatiza este proceso: dado un modelo del sistema y un criterio de cobertura, un algoritmo de generación produce un conjunto de casos de prueba que satisface dicho criterio.

Utting et al. [26] proponen una taxonomía que clasifica los enfoques de MBT según cinco dimensiones: la notación del modelo (máquinas de estados, autómatas, modelos de flujo de datos), el criterio de selección de pruebas (cobertura estructural, cobertura de requisitos, selección aleatoria), la tecnología de generación (model checking, ejecución simbólica, búsqueda), la ejecución de las pruebas (*online* versus *offline*), y el nivel de automatización del proceso completo.

En el contexto de sistemas reactivos—sistemas que interactúan con su entorno mediante estímulos y respuestas—Tretmans [25] formaliza la relación de conformidad *ioco* (*input-output conformance*) como criterio para determinar si una implementación se comporta de acuerdo con su especificación. Bajo esta teoría, el modelo del sistema se expresa como un Sistema de Transición Etiquetado (*Labelled Transition System*, LTS), y los casos de prueba se generan como secuencias de estímulos y observaciones que verifican la conformidad de la implementación con respecto al modelo.

El atractivo fundamental de MBT reside en la posibilidad de generar automáticamente grandes conjuntos de casos de prueba con garantías formales de cobertura, lo cual resulta particularmente valioso en dominios donde la

[1]Cucumber es un *framework* de código abierto que ejecuta especificaciones Gherkin como pruebas automatizadas, vinculando cada paso del escenario con código de prueba mediante expresiones regulares [3].

exhaustividad de las pruebas es crítica. Sin embargo, la aplicación de MBT requiere la existencia de un modelo formal del sistema, cuya construcción constituye en sí misma un desafío significativo. En la tesis doctoral de Brandán Briones [2] se desarrollan teorías para MBT que abordan aspectos de tiempo real y cobertura en este tipo de sistemas. Entre los formalismos utilizados como base para MBT se encuentran los Sistemas de Transición Simbólicos, que se presentan en la sección siguiente.

2.3. Sistemas de Transición Simbólicos

Los Sistemas de Transición Simbólicos (STS, *Symbolic Transition Systems*) constituyen una extensión de los Sistemas de Transición Etiquetados (LTS) que incorpora una representación explícita de datos y flujo de control dependiente de datos [8]. Mientras que en un LTS los valores de datos se representan de manera concreta—lo cual conduce a una explosión del espacio de estados cuando los dominios de datos son grandes o infinitos—los STS operan a nivel simbólico, utilizando variables y fórmulas de lógica de primer orden para expresar condiciones sobre los datos.

Formalmente, un STS se define como una tupla $\langle L, l_0, V, \iota, I, \Lambda, \rightarrow \rangle$, donde [7]:

- L es un conjunto finito de *localidades* (estados de control).
- $l_0 \in L$ es la localidad inicial.
- V es un conjunto de *variables de localidad* que representan el estado del sistema.
- ι es una valuación inicial que asigna valores a las variables en V.
- I es un conjunto de *variables de interacción* utilizadas para el intercambio de datos con el entorno.
- Λ es un conjunto de *gates* (etiquetas de interacción) que representan los canales de comunicación.
- $\rightarrow \subseteq L \times \Lambda \times \Phi \times F \times L$ es la relación de *switches* (transiciones), donde Φ denota guardas lógicas y F funciones de actualización de variables.

Cada switch representa una transición entre dos localidades, condicionada por una guarda (una fórmula lógica sobre las variables de localidad y de interacción) y acompañada por una función de actualización que modifica el

estado. Los gates se clasifican en gates de entrada (que reciben estímulos del entorno) y gates de salida (que emiten respuestas), reflejando la naturaleza reactiva de los sistemas modelados.

La ventaja de esta representación simbólica es doble. En primer lugar, evita la enumeración explícita de valores de datos, lo cual permite modelar sistemas con dominios infinitos sin incurrir en explosión de estados. En segundo lugar, preserva la información semántica de las restricciones sobre los datos, que puede ser explotada durante la generación de casos de prueba. Frantzen et al. [7] definen la relación de conformidad simbólica *sioco*, que extiende la relación *ioco* de Tretmans al nivel simbólico, proporcionando una base formal para la generación de pruebas directamente desde especificaciones STS.

Los conceptos de STS resultan esenciales para comprender el lenguaje IBDD presentado en la sección siguiente, ya que IBDD adopta directamente la terminología y estructura de los STS—variables, gates, guardas y asignaciones—como elementos de su gramática formal.

2.4. Lenguaje Intermedio de BDD

El Lenguaje Intermedio de BDD (IBDD, *Intermediate Behavior-Driven Development*) fue propuesto por Zameni et al. [30] como puente formal entre los escenarios textuales BDD y los modelos formales requeridos por las técnicas de MBT. IBDD conserva la estructura Given-When-Then de BDD pero incorpora elementos de los Sistemas de Transición Simbólicos, añadiendo la precisión formal necesaria para habilitar transformaciones automáticas hacia modelos de prueba.

La motivación de IBDD radica en que los escenarios BDD textuales presentan dos limitaciones fundamentales para su uso en MBT: ambigüedad inherente al lenguaje natural e incompletitud de la información necesaria para construir modelos formales. Elementos como las variables de estado del sistema, los tipos de datos, las guardas lógicas y los canales de interacción no están explícitos en el texto Gherkin y deben ser inferidos por un experto del dominio. IBDD hace explícita esta información mediante una gramática híbrida que combina notación EBNF con estructuras matemáticas de lógica de primer orden.

2.4.1. Información Específica del Dominio

IBDD introduce el concepto de Información Específica del Dominio (DSI, *Domain-Specific Information*), definida como una tupla $(\mathcal{S}, \mathcal{F}, \mathcal{V}, \mathcal{G})$, donde [30]:

- $\mathcal{S}$ es el conjunto de *sorts* (tipos de datos), que incluye tipos primitivos (Bool, Int), tipos compuestos, enumeraciones y tipos lista.
- $\mathcal{F}$ es el conjunto de funciones definidas sobre los sorts.
- $\mathcal{V}$ es el conjunto de variables tipadas, particionado en tres subconjuntos disjuntos: variables *globales* (representan el estado persistente del sistema), variables *locales* (con alcance limitado al escenario) y variables de *interacción* (utilizadas para el intercambio de datos en una transición).
- $\mathcal{G}$ es el conjunto de *gates*, particionado en gates de entrada $\mathcal{G}_i$ (iniciados por el entorno, prefijados con `?`) y gates de salida $\mathcal{G}_o$ (iniciados por el sistema, prefijados con `!`).

2.4.2. Gramática IBDD

La gramática IBDD estructura cada escenario en tres cláusulas:

- **Given**: declara variables locales e introduce una guarda de entrada (*input guard*), que es una conjunción de predicados sobre variables globales y locales representando la precondición del escenario.
- **When**: consiste en uno o más *switches*, cada uno compuesto por una interacción (un gate con sus variables de interacción), una condición (un predicado booleano) y una asignación (una función de actualización de variables).
- **Then**: consiste en cero o más switches de salida seguidos de una guarda de salida (*output guard*) que define la postcondición del escenario.

A modo de ejemplo, el siguiente término IBDD representa un escenario del dominio de impresora industrial donde un trabajo de impresión se mueve a la lista de trabajos completados [30]:

Listing 2.2: Ejemplo de un término IBDD para un escenario de impresora industrial.

```
GIVEN CJ [is_in_list(CJ, SJL) ∧CJ.type = Production]
WHEN !printstart.cj
      cj.id = CJ.id ∧cj.state = Printing
      CJ.state := cj.state
     !printcomplete.cj
      cj.id = CJ.id ∧cj.state = Completed
      CJ.state := cj.state, PJL := add(CJ, PJL)
THEN [is_in_list(CJ, PJL)]
```

En este ejemplo, `CJ` es una variable local que representa un trabajo de control, `SJL` y `PJL` son variables globales que representan las listas de trabajos programados e impresos, y `!printstart` y `!printcomplete` son gates de salida del sistema. Las condiciones verifican la identidad y el estado del trabajo, mientras que las asignaciones actualizan el estado de las variables.

2.4.3. Pipeline de Transformación

IBDD se sitúa en el centro de un pipeline de transformación que conecta los escenarios BDD con la generación automática de casos de prueba. Este pipeline fue desarrollado progresivamente por Zameni et al. y consta de las siguientes etapas:

1. **BDD → IBDD**: traducción de escenarios Gherkin al formalismo IBDD, definida por Zameni et al. [30] en 2024 junto con la especificación del lenguaje. Esta etapa ha sido realizada de forma manual, ya que requiere experiencia especializada en la sintaxis IBDD y en teoría de sistemas de transición.

2. **IBDD → BDDTS**: transformación formal de términos IBDD a Sistemas de Transición BDD (*BDD Transition Systems*), una variante de STS que particiona las variables de estado en locales y globales, y añade una localidad objetivo junto con guardas de entrada y salida. Esta transformación fue formalizada por Zameni et al. [30] en 2024, aunque una versión preliminar—sin la etapa intermedia de IBDD—había sido presentada en 2023 [31].

3. **BDDTS → STS**: transformación de los BDDTS a Sistemas de Transición Simbólicos. Zameni et al. [30] esbozaron tres enfoques posibles para esta transformación en 2024, y posteriormente formalizaron uno de ellos—la construcción *Check-Retrieve Symbolic System*—en 2025 [32].

4. **STS → Tests**: aplicación de algoritmos de generación automática de casos de prueba sobre los STS resultantes. Esta etapa se apoya en algoritmos preexistentes desarrollados por Frantzen et al. [7] y van den Bos y Tretmans [1].

Zameni et al. [32] extienden este pipeline en 2025 con un operador formal de composición secuencial de BDDTS, que permite construir modelos complejos a partir de escenarios individuales. Los autores demuestran la corrección del operador mediante pruebas de solidez y completitud, y reportan su aplicación a un caso de estudio industrial de sistemas de iluminación aeroportuaria. Cabe

notar que, hasta el momento, la primera etapa de este pipeline—la traducción de escenarios Gherkin a IBDD—no ha sido abordada de forma automática en la literatura. La automatización de esta etapa requiere un mecanismo capaz de interpretar texto en lenguaje natural y generar representaciones formales, capacidad que ofrecen los Modelos de Lenguaje Grandes.

2.5. Modelos de Lenguaje Grandes

Los Modelos de Lenguaje Grandes (LLMs, *Large Language Models*) son redes neuronales con miles de millones de parámetros, entrenadas sobre corpus masivos de texto para predecir la siguiente secuencia de tokens en un contexto dado. Su arquitectura se fundamenta en el *Transformer* [27], que introdujo el mecanismo de *self-attention* como alternativa a las redes recurrentes, permitiendo capturar dependencias a larga distancia en secuencias de texto de forma paralelizable y eficiente.

A partir de esta arquitectura base, modelos sucesivos han demostrado capacidades emergentes que trascienden la generación de texto: comprensión semántica profunda, razonamiento sobre estructuras formales, y generación de código en múltiples lenguajes de programación. Entre los modelos más representativos del estado del arte se encuentran GPT-4o de OpenAI [18], Claude de Anthropic y Llama de Meta, que difieren en sus orígenes—propietarios de uso general, orientados a razonamiento, y de código abierto, respectivamente—pero comparten la capacidad de procesar y generar texto estructurado con alto grado de coherencia. La selección del modelo más adecuado para una tarea específica requiere una evaluación comparativa que considere factores como la corrección de las salidas, la consistencia entre ejecuciones y la capacidad de adherirse a formatos formales.

2.5.1. Aplicaciones en Ingeniería de Software

La aplicación de LLMs en ingeniería de software ha experimentado un crecimiento notable. Jiang et al. [12] presentan un relevamiento exhaustivo que identifica las principales tareas en las que los LLMs han demostrado efectividad: generación de código a partir de descripciones en lenguaje natural, completado automático de código, detección y corrección de errores, generación de pruebas, y traducción entre lenguajes de programación. Herramientas como GitHub Copilot ilustran la adopción industrial de estas capacidades.

En el contexto específico de BDD, Karpurapu et al. [13] evalúan el uso de LLMs para la formulación automática de pruebas de aceptación. Sus hallazgos indican que la técnica de *few-shot prompting*—donde se incluyen

ejemplos resueltos de la tarea como parte de la entrada al modelo—mejora significativamente la precisión de las traducciones generadas en comparación con estrategias de *zero-shot prompting*, en las que el modelo recibe únicamente la instrucción sin ejemplos de referencia.

2.5.2. Limitaciones

A pesar de sus capacidades, los LLMs presentan limitaciones relevantes para tareas que requieren corrección formal. Los modelos pueden generar salidas sintácticamente incorrectas, introducir *hallucinations* (contenido plausible pero factualmente incorrecto), y exhibir inconsistencias entre ejecuciones sucesivas. Estas limitaciones son particularmente críticas cuando el objetivo es producir representaciones formales, donde un error sintáctico puede invalidar completamente la salida generada. Estas limitaciones motivan la necesidad de complementar los LLMs con mecanismos de verificación formal, aspecto que se abordará tras revisar los trabajos relacionados.

2.6. Trabajos Relacionados

Esta sección revisa los trabajos previos más relevantes, organizados según tres dimensiones: el pipeline formal BDD-a-MBT, el uso de técnicas de procesamiento de lenguaje natural para la automatización de BDD, y la aplicación de modelos de lenguaje grandes en este dominio.

2.6.1. Pipeline Formal BDD-a-MBT

La línea de investigación iniciada por Zameni et al. ha desarrollado progresivamente un pipeline de transformación desde escenarios BDD hasta la generación automática de casos de prueba. En su trabajo inicial [31], los autores presentaron la idea de transformar manualmente escenarios BDD en modelos formales sobre los cuales aplicar técnicas de MBT, demostrando la viabilidad del enfoque pero señalando la dificultad de automatizar la transformación debido a la gran discrepancia entre los escenarios textuales y los modelos formales. Posteriormente, Zameni et al. [30] definieron el lenguaje intermedio IBDD y formalizaron la transformación de IBDD a BDDTS y de BDDTS a STS, estableciendo así los eslabones formales de la cadena. Más recientemente [32], extendieron el framework con mecanismos de composición secuencial de BDDTS que permiten construir modelos complejos a partir de escenarios individuales. Sin embargo, la traducción inicial de escenarios Gherkin a IBDD no ha sido explorada de forma automática hasta el momento.

2.6.2. Automatización de BDD mediante NLP

La aplicación de técnicas de procesamiento de lenguaje natural a escenarios BDD tiene antecedentes previos a la era de los LLMs. Soeken et al. [23] propusieron un flujo asistido para BDD donde el sistema utiliza NLP para extraer semi-automáticamente la estructura del código de implementación y los casos de prueba a partir de escenarios textuales, estableciendo un diálogo interactivo con el usuario. Este enfoque opera a nivel de código fuente y no genera representaciones formales intermedias.

Fischbach et al. [6] desarrollaron SPECMATE, una herramienta que automatiza la creación de casos de prueba a partir de criterios de aceptación expresados en lenguaje natural. SPECMATE utiliza técnicas de NLP para extraer relaciones causales de los criterios de aceptación y generar árboles de causa-efecto, a partir de los cuales se derivan automáticamente los casos de prueba. SPECMATE no genera representaciones formales intermedias ni se integra con frameworks de MBT basados en STS.

2.6.3. LLMs para la Automatización de BDD

Un trabajo particularmente relevante es el de Karpurapu et al. [13], quienes realizaron una evaluación exhaustiva del uso de LLMs para la formulación automática de pruebas de aceptación en BDD. Sus experimentos abarcan múltiples modelos y estrategias de prompting, y sus resultados demuestran que la técnica de *few-shot prompting*—donde se proveen ejemplos de la tarea al modelo—mejora significativamente la calidad de las salidas en comparación con *zero-shot prompting*.

2.6.4. Posicionamiento del Presente Trabajo

La revisión de los trabajos previos permite situar la contribución del presente trabajo en relación con el estado del arte e identificar el espacio que este busca cubrir.

Los enfoques de NLP aplicados a BDD, como los de Soeken et al. [23] y Fischbach et al. [6], demuestran la viabilidad de aplicar técnicas de procesamiento de lenguaje natural a escenarios BDD. No obstante, estos trabajos operan a nivel de código fuente o casos de prueba directamente, sin generar representaciones formales intermedias como IBDD que puedan alimentar un pipeline de MBT.

Por otra parte, los trabajos de Karpurapu et al. [13] evidencian el potencial de estos modelos para tareas de generación estructurada en el contexto de BDD, aunque su enfoque se concentra en la dirección inversa (historias de

usuario a Gherkin) y no contempla la integración con mecanismos de validación sintáctica determinísticos basados en gramáticas formales.

En cuanto al pipeline formal, Zameni et al. [30, 31, 32] han formalizado las transformaciones desde IBDD hasta la generación de casos de prueba; sin embargo, la primera etapa—la traducción de escenarios Gherkin a IBDD—no parece haber sido abordada de forma automática en los trabajos existentes.

El presente trabajo se sitúa en la convergencia de estas líneas de investigación, proponiendo un enfoque que articula la capacidad de comprensión semántica de los LLMs con la validación sintáctica determinística mediante gramáticas formales, aplicado específicamente a la traducción de escenarios Gherkin al formalismo IBDD. El componente de validación formal de este enfoque se sustenta en las técnicas de análisis sintáctico que se describen a continuación.

2.7. Análisis Sintáctico y Gramáticas Formales

El análisis sintáctico (*parsing*) es el proceso de verificar si una cadena de texto pertenece al lenguaje definido por una gramática formal, y de construir una representación estructurada (típicamente un árbol de derivación) que refleje la estructura jerárquica de la entrada.

2.7.1. Notación EBNF

La gramática IBDD se especifica utilizando la notación EBNF (*Extended Backus-Naur Form*), propuesta por Niklaus Wirth [29] como extensión de la notación BNF (*Backus-Naur Form*) introducida por John Backus en 1959 para describir la sintaxis de ALGOL 60. Wirth observó que la diversidad innecesaria de notaciones sintácticas en la literatura dificultaba la comunicación, y propuso EBNF como notación unificada que incorpora operadores de expresiones regulares: corchetes `[ ]` para secuencias opcionales, llaves { } para repeticiones, y paréntesis `( )` para agrupamiento. Estas extensiones permiten expresar gramáticas de manera más compacta, evitando el uso intensivo de recursión para representar repeticiones simples.

2.7.2. Algoritmo de Earley

El algoritmo de Earley [5] es un algoritmo de análisis descendente basado en programación dinámica que opera sobre gramáticas libres de contexto arbitrarias. A diferencia de los analizadores LR o LALR, que requieren

gramáticas restringidas a subclases específicas, el algoritmo de Earley acepta cualquier gramática libre de contexto sin necesidad de transformaciones previas.

El algoritmo procesa la cadena de entrada de izquierda a derecha, manteniendo un conjunto de *items* (reglas de producción parcialmente reconocidas) organizados en columnas correspondientes a cada posición de la entrada. En cada paso, tres operaciones fundamentales avanzan el reconocimiento: *prediction* (predice qué reglas pueden aplicarse), *scanning* (consume un terminal de la entrada) y *completion* (propaga el reconocimiento de un no-terminal completado). Su complejidad temporal es $O(n^3)$ en el caso general, $O(n^2)$ para gramáticas no ambiguas, y $O(n)$ para gramáticas LR(k), donde n es la longitud de la entrada.

Una ventaja significativa del algoritmo de Earley es su capacidad para operar con gramáticas que no se ajustan a las restricciones de los analizadores LALR convencionales, como aquellas que combinan elementos EBNF con estructuras matemáticas de lógica de primer orden [5, 11]. Además, su flexibilidad permite modificar y extender gramáticas sin necesidad de reestructurarlas para satisfacer requisitos de analizadores más restrictivos [11].

2.7.3. Lark como Herramienta de Análisis

Lark [16] es una biblioteca de análisis sintáctico para Python que soporta múltiples algoritmos, incluyendo Earley y LALR(1). Lark permite definir gramáticas en una variante de EBNF y genera automáticamente el analizador correspondiente, proporcionando árboles de derivación tipados y mensajes de error detallados que facilitan el diagnóstico de fallos sintácticos.

Capítulo 3

Enfoque y Metodología

3.1. Selección del Modelo de Lenguaje

La tarea central de este trabajo—traducir escenarios BDD en lenguaje natural a representaciones formales IBDD—requiere un modelo de lenguaje con capacidades de comprensión semántica, generación estructurada y consistencia sintáctica. Esta sección describe el proceso de evaluación comparativa que condujo a la selección del modelo utilizado.

3.1.1. Metodología de Evaluación

Se evaluaron tres modelos de última generación: GPT-4o de OpenAI, Claude Sonnet 4 de Anthropic y Llama 3.3 70B de Meta. La selección de estos modelos responde a la diversidad de sus orígenes—propietario de uso general, propietario orientado a razonamiento, y de código abierto—y a su representatividad del estado del arte al momento de la evaluación.

La evaluación se realizó sobre un escenario de *e-commerce* que involucra interacciones típicas de sistemas web: gestión de productos, carritos de compra, inventario y sesiones de usuario. Este escenario presenta desafíos característicos de la traducción BDD→IBDD, ya que requiere identificar variables globales (inventario, carrito) y locales (producto, cantidad), inferir gates de interacción apropiados (`?click`, `?input`, `!add`, `!update`), establecer guardas lógicas para validaciones y definir asignaciones de estado correctas.

Escenario de Prueba

El escenario BDD utilizado para la evaluación fue el siguiente:

```
Scenario: A product is added to shopping cart when user selects it
  Given a product is available in inventory
```

```
And the user is logged in
When the user clicks add to cart button
And the user specifies quantity
Then the product is added to the shopping cart
And the inventory is updated
```

Diseño del Prompt

Para la evaluación se diseñó un prompt preliminar que incorpora la definición completa de la gramática IBDD en notación EBNF, una explicación de los elementos clave del lenguaje (variables locales y globales, gates, guards y assignments), un ejemplo canónico de referencia del dominio de impresora industrial extraído de los trabajos de Zameni et al. [30], e instrucciones para generar tres opciones de traducción con enfoques diferenciados. El diseño definitivo del prompt y sus componentes se desarrollan en detalle en la Sección 3.3. Los criterios de evaluación abarcaron la granularidad de las interacciones, el modelado de variables, el nivel de detalle y la estructuración de estados. Este prompt se aplicó de manera idéntica a los tres modelos para garantizar la comparabilidad de los resultados. La solicitud de tres opciones por modelo permitió evaluar tanto la capacidad de traducción como la diversidad de los enfoques propuestos.

3.1.2. Resultados de la Evaluación Comparativa

Se obtuvieron nueve traducciones en total (tres opciones por cada modelo). La Tabla 3.1 resume los resultados según los criterios evaluados.

Tabla 3.1: Comparación de modelos para la traducción BDD→IBDD

Criterio	**GPT-4o**	**Claude S. 4**	**Llama 3.3**
Corrección sintáctica	Alta	Media	Baja
Consistencia	Alta	Media	Baja
Diversidad de enfoques	Alta	Alta	Baja
Preservación semántica	Alta	Alta	Media
Calidad de justificaciones	Alta	Media	Baja

GPT-4o

GPT-4o demostró el desempeño más consistente entre los tres modelos. Las tres opciones generadas respetan la gramática IBDD sin desviaciones

sintácticas, y cada una explora un paradigma técnico distinto: atómico/granular, compuesto y concurrente con reservas. El siguiente fragmento ilustra la Opción 2 (enfoque compuesto):

```
GIVEN PROD, USER [is_available(PROD, INV) ∧is_logged_in(USER)]
WHEN
    ?addToCart.prod_qty, payload
        payload.product = PROD ∧payload.qty > 0
        (PROD, QTY) := (payload.product, payload.qty)
THEN
    !persist.cart_and_inventory, result
        result.cart_after = addItem(S_CART, PROD, QTY) ∧
        result.inv_after = INV[PROD] - QTY
        CART := result.cart_after;
        INV := updateInventory(INV, PROD, QTY)
    [is_in_cart(PROD, CART) ∧INV[PROD] = INV_before(PROD) - QTY]
```

Esta traducción modela la interacción del usuario como un gate de entrada (`?addToCart`), define guardas que verifican la disponibilidad del producto y la validez de la cantidad, y establece asignaciones que reflejan las modificaciones al carrito y al inventario. El propio modelo identifica correctamente los compromisos de diseño entre las opciones, señalando por ejemplo que la tercera opción puede resultar sobredimensionada. En efecto, algunas opciones tienden a introducir niveles de complejidad que, si bien son técnicamente válidos, exceden lo requerido por el escenario, y las explicaciones que acompañan cada traducción resultan a veces verbosas.

Claude Sonnet 4

Claude exhibió creatividad conceptual en sus traducciones, con una progresión deliberada de simplicidad a complejidad entre las tres opciones, una documentación detallada de las variables utilizadas y guardas lógicas bien estructuradas. Sin embargo, la consistencia sintáctica fue inferior a la de GPT-4o. En particular, una de las opciones introduce notaciones como `BTN_STATE := clicked` que no se ajustan a las convenciones de la gramática IBDD de referencia:

```
GIVEN P, U, Q [P.available = true ∧U.logged_in = true]
WHEN ?click.add_to_cart_btn
     U.logged_in = true ∧P.available = true
     BTN_STATE := clicked
```

Si bien la traducción es conceptualmente correcta, estas desviaciones

sintácticas requerirían corrección posterior para ser procesadas por el analizador IBDD.

Llama 3.3 70B

Llama 3.3 presentó las mayores limitaciones. Si bien las traducciones son breves y no contienen errores conceptuales graves, las tres opciones presentan errores sintácticos consistentes, ninguna modela la actualización de inventario (perdiendo información del escenario original), las justificaciones de diseño carecen de sustento técnico, y se confunde la dirección de los gates de interacción. El siguiente fragmento ilustra estos problemas:

```
GIVEN prod [true], user logged_in
WHEN !select_prod, ?spec_quantity
     true
     prod := p
THEN !add_to_cart.sc
     sc.product = p ∧sc.quantity = q
     SC := add(p, sc), Q := spec_quantity
     [is_in_SC(p, sc) ∧sc.product = p]
```

En esta traducción, el gate `!select_prod` (gate de salida) se utiliza para modelar una acción del usuario, cuando la semántica de IBDD establece que las acciones del usuario corresponden a gates de entrada (`?`). La combinación de un gate de salida con uno de entrada en la cláusula `WHEN` viola la estructura esperada del lenguaje.

3.1.3. Validación con Literatura Existente

Los resultados obtenidos son consistentes con hallazgos de la literatura. Karpurapu et al. [13] evaluaron varios LLMs (GPT-3.5, GPT-4, Llama-2-13B, PaLM-2) para la generación automática de tests de aceptación BDD a partir de historias de usuario. En su estudio, los modelos GPT exhibieron menor cantidad de errores sintácticos y mejor adherencia a las reglas y mejores prácticas de Gherkin al generar archivos *feature*, mientras que Llama-2-13B mostró dificultades significativas en evaluaciones *zero-shot*, con 130 instancias de palabras clave Gherkin ausentes y 335 instancias de patrones restringidos. Sus resultados también confirman que la técnica de *few-shot prompting* produce mayor precisión que *zero-shot*.

Aunque el trabajo de Karpurapu et al. aborda la dirección inversa a la de este trabajo (historias de usuario → Gherkin, frente a Gherkin → IBDD), los patrones observados son similares: los modelos GPT mantienen consistencia

sintáctica superior, Llama presenta dificultades recurrentes con formalismos sintácticos, y la provisión de ejemplos—como el caso de la impresora industrial incluido en el prompt de traducción—mejora los resultados. Esta convergencia refuerza la validez de la selección realizada.

3.1.4. Decisión Final

Sobre la base de los resultados experimentales y su consistencia con la literatura, se seleccionó la familia de modelos GPT-4o de OpenAI como motor de lenguaje para el sistema de traducción. El modelo demostró corrección sintáctica en todas las traducciones generadas, alta uniformidad de calidad entre ejecuciones y capacidad de explorar enfoques alternativos genuinos. Si bien se trata de un modelo propietario que requiere acceso mediante la API de OpenAI, cuyo uso se factura por cantidad de tokens procesados, esta limitación se considera aceptable en el contexto del presente trabajo.

No obstante, una decisión de diseño fundamental del sistema es su independencia respecto del modelo y del proveedor de lenguaje utilizados. Si bien GPT-4o demostró el mejor desempeño en la evaluación comparativa, la arquitectura del sistema abstrae la comunicación con el LLM mediante una capa de abstracción que parametriza tanto el modelo como el proveedor como variables de configuración. Esta abstracción permite intercambiar el modelo subyacente, ya sea entre versiones de un mismo proveedor o entre proveedores distintos, sin modificar ningún otro componente del pipeline.

Esta decisión responde a múltiples consideraciones. En primer lugar, la **soberanía de datos**: en contextos institucionales y académicos, el envío de información a servicios externos puede estar sujeto a restricciones de privacidad, regulaciones de protección de datos o políticas internas que impidan el uso de APIs propietarias en la nube. La posibilidad de ejecutar el sistema con modelos de código abierto desplegados localmente (como Llama u otros modelos compatibles a través de Ollama) permite operar el pipeline sin que los datos abandonen la infraestructura de la institución. En segundo lugar, la **reproducibilidad y continuidad**: los modelos propietarios están sujetos a actualizaciones, discontinuaciones y cambios en las condiciones de servicio por parte de sus proveedores; un sistema acoplado a un modelo específico quedaría obsoleto ante cualquiera de estos eventos. En tercer lugar, la **evolución del estado del arte**: el campo de los LLMs avanza con rapidez, y modelos de código abierto que hoy presentan limitaciones podrían alcanzar o superar el desempeño de los modelos propietarios en el corto plazo. Un sistema agnóstico al modelo puede incorporar estos avances de forma transparente, sin requerir rediseño.

En la práctica, el sistema soporta actualmente dos proveedores: OpenAI

(para modelos propietarios como GPT-4o) y Ollama (para modelos de código abierto ejecutados localmente). La selección se realiza mediante parámetros de línea de comandos, lo que facilita la evaluación comparativa entre modelos y la adaptación a diferentes contextos de despliegue. La Sección 4.3 describe la implementación de esta abstracción.

3.2. Selección de la Herramienta de Análisis Sintáctico

La validación sintáctica de las traducciones IBDD generadas por el LLM requiere una herramienta de análisis sintáctico (*parsing*) capaz de verificar conformidad con la gramática formal IBDD. El analizador debe resolver ambigüedades sintácticas, proveer reportes de errores precisos, integrarse con Python—lenguaje de implementación del sistema—, ofrecer confiabilidad para uso en investigación, y permitir la especificación declarativa de la gramática en un formato legible.

3.2.1. Lark: Características y Algoritmos

Lark [14] es una biblioteca de análisis sintáctico para Python que ha alcanzado amplia adopción tanto en la industria como en la academia. Implementa tres algoritmos principales de análisis. El algoritmo Earley logra rendimiento lineal en casos favorables y puede procesar gramáticas ambiguas, lo que lo hace adecuado para prototipado y lenguajes de estructura compleja; su principal limitación es un rendimiento aproximadamente diez veces inferior al de LALR en gramáticas no ambiguas [9]. El algoritmo LALR ofrece el mejor rendimiento para gramáticas no ambiguas bien definidas, aunque su *lexer* contextual puede presentar fallos en ciertos casos de ambigüedad [15]. El algoritmo CYK presenta limitaciones que incluyen un defecto reportado que causa bucles infinitos con gramáticas recursivas [15], por lo que no resulta apropiado para uso fuera de contextos teóricos.

Se consideraron otras bibliotecas de análisis sintáctico para Python antes de seleccionar Lark. PLY (*Python Lex-Yacc*) implementa exclusivamente LALR(1) y no soporta gramáticas ambiguas, lo que lo descarta para la gramática IBDD que combina estructuras EBNF con expresiones de lógica de primer orden. ANTLR4, si bien es una herramienta robusta con soporte para múltiples lenguajes, genera analizadores LL(*) que tampoco manejan ambigüedades de forma nativa, y su arquitectura basada en Java introduce una dependencia externa significativa. Las bibliotecas basadas en *Parsing Expression Grammars* (PEG), como pyparsing y TatSu, resuelven las ambigüedades

mediante prioridad ordenada en lugar de explorar todas las derivaciones posibles, lo cual puede descartar silenciosamente interpretaciones válidas de la gramática IBDD. Lark, en cambio, ofrece el algoritmo Earley con soporte nativo para gramáticas ambiguas, especificación declarativa en EBNF y mensajes de error con posición exacta del fallo. Según análisis independientes [10], Lark se posiciona favorablemente frente a estas alternativas en el balance entre flexibilidad, rendimiento y facilidad de uso.

3.2.2. Adopción Industrial y Académica

Lark está clasificado como *Production/Stable* en PyPI, cuenta con más de 5 500 estrellas en GitHub y mantiene desarrollo activo [20]. Su adopción en sistemas de escala empresarial respalda su confiabilidad: Dailymotion procesa más de 100 millones de llamadas API GraphQL diarias utilizando Lark para el análisis de *GraphQL Schema Definition Language* [**dailymotion˙graphql**]; Vyper, el lenguaje de contratos inteligentes para Ethereum, emplea Lark para el análisis de su sintaxis [28]; y Poetry, el gestor de dependencias de Python, depende de Lark para procesar archivos de configuración [21].

En el ámbito académico, Hundvin [**hundvin2022code**] desarrolló en la Universidad de Bergen una tesis de maestría sobre generación de fragmentos de código a partir de especificaciones de gramática Lark, contribuyendo algoritmos para producir código sintácticamente correcto desde gramáticas formales. Este precedente demuestra la adecuación de Lark para investigación en análisis sintáctico y validación de lenguajes.

3.2.3. Manejo de Gramáticas Ambiguas

La gramática IBDD presenta ambigüedades inherentes a su diseño. En particular, la regla del *switch* acepta una secuencia flexible de guardas, expresiones y asignaciones tras la interacción, lo que significa que una construcción como `[true]` puede interpretarse como una guarda con valor booleano o como una asignación trivial encerrada entre corchetes, y que una expresión de la forma `x = 5` admite lectura tanto como asignación como comparación de igualdad. Un analizador LALR requeriría desambiguar estas construcciones mediante transformaciones de la gramática que sacrificarían su legibilidad; el algoritmo Earley, en cambio, las resuelve de forma automática.

El analizador Earley de Lark puede almacenar eficientemente todas las derivaciones posibles ante una ambigüedad [14]. La biblioteca ofrece tres estrategias de resolución: el modo `resolve` (predeterminado) elige automáticamente la derivación más simple; el modo `explicit` retorna todas las derivaciones para selección manual; y el modo `forest` retorna el bosque de análisis completo. En

el presente trabajo se utiliza el modo `resolve`, que resulta adecuado para las ambigüedades de la gramática IBDD ya que la derivación más simple coincide con la interpretación semánticamente correcta en todos los casos observados.

3.2.4. Decisión: Earley en Lark

Se seleccionó Lark con el algoritmo Earley como herramienta de análisis sintáctico. El algoritmo Earley ofrece la flexibilidad necesaria para manejar la estructura anidada y las potenciales ambigüedades de la gramática IBDD, permite iteración ágil durante el desarrollo de dicha gramática, y cuenta con respaldo tanto industrial como académico. La gramática se especifica en formato EBNF, lo que facilita su lectura y mantenimiento, y el analizador provee información detallada sobre violaciones sintácticas que resulta útil para el ciclo de corrección iterativa.

El compromiso de rendimiento respecto a LALR—aproximadamente un orden de magnitud más lento—se considera aceptable dado que el volumen de escenarios a procesar es moderado y la corrección es prioritaria sobre la velocidad en esta fase de investigación. Para un sistema de producción futuro, la migración a LALR sería recomendable una vez estabilizada la gramática IBDD.

3.3. Diseño del Prompt de Traducción

El diseño del prompt constituye un componente central del sistema, ya que determina la capacidad del modelo de lenguaje para producir traducciones IBDD sintáctica y semánticamente correctas. Se adoptó una estrategia de *few-shot prompting* con instrucciones estructuradas, organizada en cinco componentes que proveen al modelo con el contexto completo necesario para la tarea de traducción.

3.3.1. Estructura del Prompt

El prompt se organiza en cinco secciones. La primera establece el contexto general, indicando que se traducirán escenarios Gherkin a IBDD y que este lenguaje intermedio agrega precisión formal mediante sistemas de transición simbólicos. La segunda sección incluye la gramática IBDD completa en notación EBNF, junto con la descripción de los elementos clave del lenguaje: variables locales y globales, variables de interacción, gates de entrada, denotados con el prefijo `?`, y gates de salida, denotados con el prefijo `!`, guardas booleanas y asignaciones, así como el repertorio de operadores aceptados en

notación Unicode y ASCII. La tercera sección provee ejemplos de traducción resueltos con explicaciones del razonamiento aplicado. La cuarta sección especifica reglas de traducción para cada cláusula (Given, When, Then) y convenciones de nomenclatura. La quinta sección define el formato de entrada y salida esperado y las restricciones de formato del resultado.

La inclusión de la gramática formal directamente en el prompt responde a un requisito fundamental: el modelo debe conocer la sintaxis exacta del lenguaje objetivo para generar traducciones conformes. Sin esta especificación, las traducciones presentarían las desviaciones sintácticas observadas durante la evaluación comparativa de modelos, donde incluso GPT-4o introducía variaciones cuando el prompt no era suficientemente explícito respecto a las reglas de formato.

3.3.2. Aprendizaje por Ejemplos

Se adoptó la técnica de *few-shot prompting* con cuatro ejemplos canónicos que cubren los patrones más frecuentes de la traducción BDD→IBDD. Los primeros tres ejemplos provienen del dominio de impresora industrial, extraído de los trabajos de Zameni et al. [30], y el cuarto del dominio de comercio electrónico. Esta selección de dos dominios distintos busca que el modelo generalice los patrones de traducción en lugar de memorizar un formato específico de dominio.

El primer ejemplo ilustra un escenario con una única variable local declarada en la cláusula Given, precondición trivial (`[true]`), un gate de entrada en When y un gate de salida en Then con postcondición de verificación. El segundo ejemplo presenta variables locales con guardas no triviales y dos switches secuenciales en When que modelan un proceso de dos etapas. El tercer ejemplo muestra un escenario sin variables locales donde la cláusula Given contiene únicamente una guarda sobre el estado global del sistema. El cuarto ejemplo introduce múltiples variables de interacción y asignaciones compuestas que actualizan más de un atributo del estado.

Cada ejemplo incluye el escenario Gherkin original en formato JSON, la traducción IBDD correspondiente y una explicación del razonamiento que justifica las decisiones de traducción. Esta estructura permite que el modelo observe no solo el resultado esperado sino también el proceso inferencial que lo produce. La elección de *few-shot* frente a *zero-shot* es consistente con los hallazgos de Karpurapu et al. [13], quienes reportaron que la provisión de ejemplos mejora significativamente la precisión de los LLMs en tareas de generación de formalismos BDD.

3.3.3. Reglas de Traducción

El prompt especifica reglas explícitas para la traducción de cada cláusula Gherkin. Para la cláusula Given, el modelo debe identificar si el texto menciona entidades que constituyen variables locales (por ejemplo, "a job file" se traduce a la variable local `JF`) o si describe condiciones sobre el estado global del sistema, que se modelan como guardas sin declaración de variables. Múltiples condiciones se combinan mediante el operador de conjunción lógica y la guarda resultante se encierra entre corchetes.

Para la cláusula When, las acciones del usuario se modelan como gates de entrada (`?`) y las acciones del sistema como gates de salida (`!`). Cuando el texto contiene múltiples acciones conectadas por "AND", cada acción se traduce como un switch independiente. Cada switch debe incluir una condición entre corchetes y, opcionalmente, asignaciones que reflejen cambios de estado.

Para la cláusula Then, las verificaciones del estado final se expresan como postcondiciones en la guarda final, mientras que las acciones explícitas del sistema se modelan como switches con gates de salida. Las convenciones de nomenclatura establecen que las variables locales se representan como abreviaturas en mayúscula (`CJ`, `JF`, `P`), las variables globales como nombres descriptivos en mayúscula (`SJL`, `PJL`), las variables de interacción en minúscula (`cj`, `jf`, `p`) y los gates como verbos descriptivos en minúscula (`submit`, `printstart`, `add_to_cart`).

3.3.4. Formato de Intercambio

Se seleccionó JSON como formato de intercambio entre el sistema y el modelo de lenguaje, tanto para la representación de los escenarios BDD de entrada como para las traducciones IBDD de salida. Esta decisión se fundamenta en la relación entre los datos de entrenamiento de los modelos de lenguaje y su desempeño en tareas de generación estructurada. Los modelos de la familia GPT fueron entrenados sobre corpus masivos que incluyen una proporción significativa de datos en formato JSON, provenientes de repositorios de código fuente, documentación de APIs web, archivos de configuración y conjuntos de datos abiertos [**placeholder:gpt-training-data**]. Dado que estos modelos observaron millones de ejemplos de JSON bien formado durante su entrenamiento, han desarrollado una familiaridad inherente con la sintaxis de este formato que se manifiesta en dos niveles. A nivel de tokenización, el algoritmo *Byte Pair Encoding* (BPE) utilizado por estos modelos ha aprendido a segmentar eficientemente las estructuras sintácticas características de JSON (llaves, corchetes, comillas, dos puntos), produciendo secuencias de tokens más compactas que las obtenidas con formatos menos representados

en los datos de entrenamiento [22]. A nivel de generación, el modelo produce JSON con mayor consistencia estructural y menor tasa de errores de formato que otros formatos de serialización, precisamente porque ha internalizado los patrones sintácticos de JSON a partir de la exposición extensiva durante el entrenamiento. En consecuencia, la elección de JSON como formato de intercambio maximiza tanto la eficiencia de tokenización—reduciendo el consumo de tokens por solicitud—como la confiabilidad de las respuestas generadas.

El formato de entrada representa cada escenario BDD como un objeto JSON con campos `id`, `domain`, `title`, `given`, `when` y `then`. El formato de salida replica esta estructura incorporando un campo adicional, `ibdd_representation`, que contiene la traducción IBDD como una cadena de texto con saltos de línea (`\n`) para separar las cláusulas del escenario. Esta correspondencia estructural entre entrada y salida facilita la trazabilidad entre el escenario original y su traducción formal, y permite que el sistema preserve los metadatos del escenario (dominio, título) a lo largo del pipeline de procesamiento.

3.4. Construcción del Dataset

La evaluación del sistema de traducción requiere un conjunto de escenarios BDD que cubra diversidad de dominios, estructuras sintácticas y niveles de complejidad. Dado que no existe un corpus público de escenarios BDD anotados con traducciones IBDD de referencia, fue necesario construir un dataset específico para este trabajo.

3.4.1. Escenarios Canónicos de Referencia

El punto de partida para la construcción del dataset fueron los escenarios canónicos provenientes de dos dominios: el dominio de impresora industrial, extraído de los trabajos de Zameni et al. [30], y el dominio de calculadora. Estos escenarios cuentan con traducciones IBDD verificadas manualmente y cubren los patrones fundamentales del lenguaje: declaración de variables locales con y sin guardas, switches con gates de entrada y salida, asignaciones simples y compuestas, y postcondiciones con funciones de verificación. Su inclusión en el dataset cumple una doble función: sirven como ejemplos de referencia dentro del prompt de traducción (Sección 3.3) y como casos de validación cuya traducción correcta es conocida de antemano.

3.4.2. Generación Sintética de Escenarios

A partir de los escenarios canónicos, se generó un dataset sintético más amplio utilizando un modelo de lenguaje. El proceso de generación consistió en proveer al modelo los escenarios canónicos como ejemplos de referencia junto con instrucciones para producir nuevos escenarios BDD en formato Gherkin, representados como objetos JSON con la estructura estándar del sistema (campos `id`, `domain`, `title`, `given`, `when`, `then`). Se solicitaron escenarios que variaran en dominio de aplicación, complejidad de las interacciones y cantidad de pasos, manteniendo la plausibilidad funcional de las situaciones descritas.

La generación sintética mediante un LLM presenta la ventaja de producir escenarios lingüísticamente diversos—con variaciones naturales en la redacción de las cláusulas Given, When y Then—que ejercitan la capacidad del sistema de traducción para manejar formulaciones heterogéneas de un mismo tipo de interacción. Esta diversidad es particularmente relevante dado que los escenarios BDD en entornos reales exhiben una variabilidad considerable en estilo y nivel de detalle según el autor. Al mismo tiempo, la generación sintética introduce el riesgo de escenarios que, si bien son gramaticalmente correctos en Gherkin, describen situaciones funcionalmente implausibles o ambiguas, lo cual requirió una revisión manual posterior del dataset generado.

Capítulo 4

Implementación

Este capítulo describe la implementación del sistema de traducción automática BDD→IBDD. Se presenta la arquitectura general del pipeline, la definición formal de la gramática IBDD en Lark, el módulo de traducción basado en LLMs con su capa de abstracción agnóstica al proveedor, el módulo de validación sintáctica y el mecanismo de corrección iterativa.

4.1. Arquitectura del Sistema

El sistema implementa un pipeline de cuatro etapas orquestado por la clase `BDDToIBDDPipeline`, definida en el módulo principal. Cada etapa consume la salida de la anterior y persiste sus resultados en archivos JSON intermedios, lo que permite ejecutar las etapas de manera independiente durante el desarrollo y la depuración. A continuación se describe brevemente cada etapa; las secciones posteriores de este capítulo desarrollan en detalle los módulos correspondientes.

1. **Traducción** (Sección 4.3). Recibe el archivo `Dataset.json` con los escenarios BDD en formato Gherkin y produce traducciones IBDD mediante un LLM, utilizando salidas estructuradas para garantizar conformidad con el esquema JSON esperado. Las traducciones se persisten incrementalmente en `output.json`.

2. **Validación sintáctica** (Sección 4.4). Cada traducción se analiza contra la gramática formal IBDD utilizando el analizador Earley de Lark. Los escenarios que no superan la validación se identifican con información detallada del error, incluyendo la posición y el tipo de violación sintáctica. Los resultados se almacenan en `parsed_ibdd_results.json`.

3. **Análisis de errores** (Sección 4.5). Los escenarios que fallaron en la validación sintáctica son procesados por un agente especializado—el *explainer*—que diagnostica la causa del error y genera una explicación estructurada con sugerencias de corrección.

4. **Reintento con correcciones** (Sección 4.5). Las explicaciones de error se inyectan en un prompt de reintento que solicita al LLM una nueva traducción informada por el diagnóstico previo. Las traducciones corregidas reemplazan únicamente los casos fallidos mediante una estrategia de fusión no destructiva, y el ciclo de validación se repite sobre los casos retraducidos.

El pipeline completo se implementa en Python. Los datos fluyen secuencialmente desde `Dataset.json` hasta los archivos de resultados finales, y todas las etapas operan sobre archivos JSON como formato de intercambio.

4.2. Definición de la Gramática IBDD

La gramática IBDD se especifica en formato EBNF compatible con Lark y constituye el componente central del módulo de validación. Su diseño captura la estructura formal del lenguaje IBDD definido por Zameni et al. [30], adaptada para manejar las variaciones de formato que genera el LLM.

4.2.1. Estructura de la Gramática

La gramática define un escenario IBDD como la secuencia de tres cláusulas obligatorias: Given, When y Then. El Listado 4.1 presenta las reglas estructurales principales.

Listing 4.1: Reglas estructurales de la gramática IBDD en Lark.

```
scenario: given when then

given: "GIVEN" [vars] guard
vars: var (COMMA var)*

when: "WHEN" switch+
then: "THEN" switch* guard

switch: interaction (guard | expr | assignment)*
interaction: gate [DOT var_list]
gate: /[!?][a-zA-Z][a-zA-Z0-9_]*/
var_list: var (COMMA var)*
```

```
guard: LBRACKET expr RBRACKET
```

La cláusula `given` acepta opcionalmente una lista de variables locales seguida de una guarda entre corchetes. La cláusula `when` requiere al menos un switch, mientras que `then` acepta cero o más switches seguidos de una guarda final que representa la postcondición. Cada switch se compone de una interacción—un gate con dirección (`?` para entrada, `!` para salida) y opcionalmente una lista de variables de interacción—seguida de una secuencia flexible de guardas, expresiones y asignaciones.

La regla `switch` merece atención particular. Su definición emplea una alternancia con repetición:

```
switch: interaction (guard | expr | assignment)*
```

Esta formulación permite acomodar las diversas formas en que el LLM estructura los switches. En la gramática teórica de IBDD, un switch sigue estrictamente el patrón *interacción – condición – asignación*; sin embargo, las traducciones generadas por el modelo pueden omitir la condición, presentar la asignación antes de la guarda, o incluir expresiones sin envolver en corchetes. Esta flexibilidad sintáctica es una decisión de diseño deliberada que prioriza la capacidad de analizar traducciones imperfectas sobre el rechazo estricto de variaciones menores.

4.2.2. Jerarquía de Expresiones

Las expresiones de la gramática se organizan en una jerarquía de precedencia que refleja las convenciones matemáticas estándar. El Listado 4.2 muestra esta jerarquía.

Listing 4.2: Jerarquía de expresiones en la gramática IBDD.

```
expr: or_expr
or_expr: and_expr ((OR | OR_SYMBOL) and_expr)*
and_expr: not_expr ((AND | AND_SYMBOL) not_expr)*
not_expr: (NOT | NOT_SYMBOL) not_expr | comparison
comparison: sum [op sum]
sum: product ((PLUS|MINUS) product)*
product: power ((STAR|SLASH|PERCENT) power)*
power: atom [CIRCUMFLEX atom]
atom: literal | var | func_call
     | prop_access | neg_number
     | LPAR expr RPAR
```

La disyunción tiene la menor precedencia, seguida por la conjunción, la negación, las comparaciones, las operaciones aritméticas de suma y resta, las de multiplicación, división y módulo, y finalmente la exponenciación. Los átomos constituyen las unidades indivisibles de las expresiones e incluyen literales booleanos (`true`, `false`) y numéricos, variables, llamadas a funciones (como `is_in_list(CJ, SJL)`), accesos a propiedades (`CJ.type`) y expresiones parentizadas.

4.2.3. Soporte de Operadores

La gramática acepta tanto notación Unicode como ASCII para los operadores lógicos y de comparación. La conjunción puede expresarse como $\wedge$ o `&&`, la disyunción como $\vee$ o `||`, y la negación como $\neg$ o `!`. De manera similar, la desigualdad admite $\neq$ o `!=`, y los operadores de orden $\leq$/`<=` y $\geq$/`>=`. Este soporte dual responde a la variabilidad observada en las salidas del LLM, que puede utilizar caracteres Unicode o sus equivalentes ASCII según el contexto de generación. Sin esta flexibilidad, traducciones semánticamente correctas serían rechazadas por el analizador debido a diferencias puramente notacionales.

4.2.4. Transformación del Árbol Sintáctico

El análisis sintáctico produce un árbol de derivación que se transforma en objetos del dominio mediante la clase `IBDDTransformer`, implementada como un transformador de Lark. La transformación mapea cada nodo del árbol a una de cinco clases de datos: `IBDDScenario` representa el escenario completo con variables locales, precondición, switches de When y Then, y postcondición; `IBDDSwitch` encapsula una interacción con su condición y asignaciones; `IBDDInteraction` modela un gate con sus variables; `IBDDAssignment` representa una asignación de valor a variable o propiedad; e `IBDDExpression` captura expresiones tipadas con sus subexpresiones en una estructura recursiva. Esta representación intermedia facilita la inspección programática de las traducciones y la generación de reportes de validación estructurados.

4.3. Módulo de Traducción

El módulo de traducción se implementa en la clase `TranslationService` y gestiona la comunicación con el LLM para producir traducciones IBDD a partir de escenarios BDD. El diseño del módulo se organiza en torno a cuatro aspectos: la abstracción del proveedor de LLM, la garantía estructural de las

respuestas mediante salidas estructuradas, la inferencia dinámica del esquema JSON, y la estrategia de procesamiento de escenarios.

4.3.1. Abstracción del Proveedor de LLM

Como se discutió en la Sección 3.1, el sistema se diseñó como agnóstico al modelo y al proveedor de lenguaje. Esta independencia se implementa mediante la clase `LLMClient`, que encapsula las diferencias entre proveedores detrás de una interfaz uniforme. El cliente recibe como parámetros el identificador del proveedor (`openai`, `ollama`), el nombre del modelo, la clave de API (cuando corresponde) y la temperatura de generación, y expone un único método `generate_json` que acepta un prompt de sistema, un prompt de usuario y un esquema JSON de respuesta.

Para el proveedor OpenAI, el cliente utiliza la biblioteca LangChain con el conector `ChatOpenAI` e invoca el mecanismo de *Structured Outputs* con esquema estricto (`strict: true`), que garantiza conformidad estructural de la respuesta. Para proveedores locales como Ollama, el cliente utiliza el conector `ChatOllama` con formato JSON y aplica validación de esquema *a posteriori* mediante la biblioteca `jsonschema`; en caso de que la respuesta no cumpla con el esquema, el cliente reintenta la solicitud hasta un máximo configurable de intentos con *exponential backoff*.

Esta arquitectura permite que el resto del pipeline (el servicio de traducción, el analizador de errores y el orquestador principal) opere sin conocimiento del proveedor subyacente. La selección del modelo y del proveedor se realiza mediante parámetros de línea de comandos (`--provider`, `--model`) o variables de entorno (`LLM_PROVIDER`, `LLM_MODEL`), lo que facilita la evaluación comparativa entre modelos y la adaptación a diferentes contextos de despliegue sin modificar el código fuente.

4.3.2. Structured Outputs de OpenAI

El sistema utiliza la funcionalidad de *Structured Outputs* de OpenAI [19], que permite especificar un esquema JSON (*JSON Schema*) como parte de la solicitud a la API. Cuando se activa el modo `json_schema` con la opción `strict: true`, la API garantiza que la respuesta del modelo se ajustará exactamente al esquema proporcionado. Esta garantía se implementa mediante *constrained decoding*: durante la generación, el modelo descarta las secuencias de tokens que violarían el esquema antes de seleccionar el siguiente token, asegurando que el resultado contenga exactamente los campos especificados con los tipos correctos.

El uso de Structured Outputs en este sistema presenta dos ventajas concretas. La primera es la eliminación de errores de formato: sin esta restricción, el modelo podría devolver JSON malformado, incluir campos adicionales no solicitados, o envolver la respuesta en bloques de código Markdown, lo cual requeriría lógica de post-procesamiento frágil y propensa a errores. La segunda ventaja es la garantía de presencia del campo `ibdd_representation`, que es donde el modelo deposita la traducción IBDD para cada escenario. Al declarar este campo como obligatorio en el esquema, se asegura que toda respuesta válida contenga la traducción solicitada.

Cabe señalar que Structured Outputs garantiza la corrección estructural del JSON pero no la corrección sintáctica del contenido IBDD. El campo `ibdd_representation` contiene una cadena de texto cuyo contenido—la traducción IBDD—requiere validación adicional mediante el analizador sintáctico descrito en la sección anterior. Esta distinción entre validación de formato (a cargo de Structured Outputs) y validación de contenido (a cargo del parser Earley) es fundamental en la arquitectura del sistema.

4.3.3. Inferencia Dinámica del Esquema

El esquema JSON que se envía a la API no se define estáticamente, sino que se infiere dinámicamente a partir de la estructura de los datos de entrada mediante el método `create_response_schema`. Este método recorre recursivamente el JSON de entrada, determina el tipo de cada campo (`string`, `integer`, `boolean`, `array`, `object`) y construye un esquema compatible con JSON Schema que replica la estructura observada. Adicionalmente, el método inyecta automáticamente el campo `ibdd_representation` de tipo `string` si este no está presente en los datos de entrada, asegurando que el esquema resultante siempre incluya el campo de salida requerido para la traducción.

Para entradas de tipo arreglo—el caso habitual, donde se procesa un conjunto de escenarios—el esquema resultante envuelve los elementos en un objeto con un campo `items` de tipo arreglo. Esta envoltura responde a un requisito técnico de la API de OpenAI, que exige que el esquema raíz sea de tipo objeto. El servicio de traducción deshace esta envoltura al recibir la respuesta, extrayendo de manera transparente el arreglo de escenarios traducidos.

La inferencia dinámica permite que el sistema se adapte a variaciones en la estructura del *dataset* de entrada sin requerir modificaciones en el código. Si se agregan campos al JSON de entrada (por ejemplo, notas o metadatos adicionales), el esquema inferido los incorporará automáticamente y el modelo los preservará en la salida.

4.3.4. Procesamiento de Escenarios

El servicio procesa los escenarios de manera individual: cada escenario BDD se envía como una solicitud independiente a la API y su traducción se persiste de manera incremental en el archivo de salida. Esta estrategia de procesamiento individual—en contraposición al envío de todos los escenarios en una única solicitud—responde a dos consideraciones.

La primera es la reducción del costo de las llamadas a la API. El envío del dataset completo en una única solicitud implicaría un prompt de entrada de gran tamaño, ya que cada escenario adicional incrementa el número de tokens de contexto. Dado que la API de OpenAI factura por cantidad de tokens procesados, tanto de entrada como de salida, y que el costo por token de salida es superior al de entrada, el procesamiento individual minimiza el contexto por solicitud y reduce el costo total. Además, las solicitudes con contextos extensos presentan mayor probabilidad de respuestas truncadas o degradadas en calidad cuando el modelo debe mantener coherencia a lo largo de una salida muy extensa.

La segunda consideración es la resiliencia ante interrupciones: dado que cada traducción se persiste de manera incremental en el archivo de salida, si el proceso se detiene por un error de red, agotamiento de la cuota de la API u otra causa, las traducciones completadas se preservan y no es necesario reprocesar todo el conjunto de datos.

El módulo implementa un mecanismo de reintentos con *exponential backoff* para manejar errores transitorios de la API, particularmente los errores de límite de tasa (código HTTP 429). Ante un error, el sistema espera un intervalo que se duplica en cada reintento ($t_i = t_{\text{base}} \cdot 2^{i-1} + \epsilon$, donde ϵ es una perturbación aleatoria uniforme), hasta un máximo de cinco intentos. Este mecanismo permite procesar conjuntos de datos extensos sin interrupciones por limitaciones temporales de la API.

La temperatura del modelo se configura en 0,7, un valor que balancea la diversidad de las traducciones con la consistencia sintáctica. Valores más bajos producirían traducciones más determinísticas pero potencialmente repetitivas ante escenarios similares; valores más altos introducirían mayor variabilidad con riesgo de desviaciones sintácticas. El modelo utilizado es configurable mediante parámetros de línea de comandos, lo que permite evaluar diferentes versiones del modelo sin modificar el código fuente.

4.4. Módulo de Validación Sintáctica

El módulo de validación instancia la clase `IBDDParser`, que encapsula el analizador Earley de Lark. El parser se configura con dos opciones relevantes: resolución automática de ambigüedades (`ambiguity='resolve'`) y propagación de posiciones (`propagate_positions=True`). La primera opción indica al analizador que seleccione automáticamente la derivación más simple ante una ambigüedad, evitando la necesidad de intervención manual. La segunda habilita la generación de mensajes de error que incluyen la línea y columna exactas donde se detectó la violación sintáctica, información útil para diagnosticar problemas en las traducciones.

Antes del análisis, cada texto IBDD se somete a un preprocesamiento que normaliza los caracteres de escape (`\n` a saltos de línea reales), elimina líneas vacías y colapsa espacios múltiples dentro de cada línea. Este preprocesamiento es necesario porque las traducciones generadas por el LLM representan los saltos de línea como secuencias de escape dentro de cadenas JSON, y el analizador Earley requiere el texto con saltos de línea reales para aplicar las reglas de la gramática que dependen de la separación de cláusulas.

La validación opera en modo por lotes: el sistema procesa secuencialmente todos los escenarios de un archivo JSON, registrando para cada uno si el análisis fue exitoso o, en caso de fallo, el mensaje de error del analizador. Los resultados se persisten en un archivo JSON que asocia cada escenario con su estado de validación (`valid: true/false`), el dominio, el título y, cuando corresponde, el mensaje de error y la representación parseada del escenario. Este informe estructurado sirve como insumo para las etapas posteriores del pipeline.

El módulo incluye además un analizador de respaldo (*fallback parser*) basado en expresiones regulares que se activa cuando el analizador Earley no logra procesar un texto IBDD. Este analizador alternativo extrae los componentes principales del escenario—variables de la cláusula Given, gates de la cláusula When y postcondición de la cláusula Then—mediante patrones de coincidencia, produciendo una representación parcial pero útil para el diagnóstico de errores.

4.5. Módulo de Corrección Iterativa

Cuando el analizador sintáctico detecta que una traducción IBDD no es conforme a la gramática, el pipeline activa un ciclo de corrección que involucra tres etapas: identificación de los casos fallidos, análisis automatizado de los errores mediante un agente explicador, y reintento de la traducción

con contexto del error. Este ciclo constituye el mecanismo central de autocorrección del sistema y opera de manera no destructiva sobre las traducciones existentes.

4.5.1. Identificación de Casos Fallidos

Al finalizar la validación sintáctica, el orquestador del pipeline cruza los resultados de validación con las traducciones originales para construir un expediente completo de cada caso fallido. Para cada escenario cuyo campo `valid` es `false`, el sistema recopila cuatro elementos: el identificador del caso, el escenario BDD original (cláusulas Given, When y Then en lenguaje natural), la traducción IBDD que generó el error, y el mensaje de error producido por el analizador Earley, que incluye la posición exacta (línea y columna) y el tipo de token inesperado. Esta información constituye el insumo que alimenta al agente explicador de errores.

4.5.2. Análisis de Errores

El análisis de errores se implementa en la clase `IBDDErrorExplainer`, un agente especializado que utiliza el LLM para interpretar los mensajes de error del parser y producir diagnósticos estructurados. A diferencia del módulo de traducción, que opera con una temperatura de 0,7, el explicador se configura con una temperatura de 0,3. Este valor más bajo favorece respuestas determinísticas y analíticas, apropiadas para una tarea de diagnóstico donde la creatividad no es deseable.

El agente explicador opera con un prompt de sistema que incluye la gramática IBDD de referencia y un catálogo de patrones de error frecuentes: corchetes faltantes en guardas o condiciones, sintaxis de gate incorrecta (confusión entre `?` y `!`), asignaciones ausentes, expresiones mal formadas, saltos de línea incorrectos y postcondiciones faltantes. Este catálogo permite que el modelo enfoque su análisis en las categorías de error más probables.

Para cada caso fallido, el explicador construye un prompt de análisis que presenta al modelo el escenario BDD original, la traducción IBDD que falló y el mensaje de error del parser, solicitando un diagnóstico estructurado. La respuesta se obtiene mediante Structured Outputs con un esquema que define cinco campos obligatorios: `error_type` (clasificación del error, por ejemplo "corchetes faltantes" o "gate inválido"), `error_location` (posición exacta dentro del texto IBDD), `explanation` (descripción de la discrepancia entre lo esperado por el parser y lo encontrado), `correction_suggestion` (fragmento IBDD corregido o sugerencia concreta) y `hints` (lista de recomendaciones adicionales). El uso de Structured Outputs garantiza que todo diagnóstico contenga estos

cinco campos con los tipos de datos correctos, facilitando su procesamiento automatizado en la etapa siguiente.

4.5.3. Reintento con Contexto de Error

El reintento de traducción utiliza un prompt especializado que extiende el prompt de traducción inicial con información del error. El prompt de reintento contiene las mismas cinco secciones del prompt original—contexto, gramática, ejemplos, reglas y formato—pero incorpora al inicio una sección adicional de análisis de error, precedida por un encabezado que indica al modelo que se encuentra en modo de corrección. Esta sección se inyecta dinámicamente mediante un *placeholder* {`error_analysis`} que el sistema reemplaza con el diagnóstico formateado del caso específico.

El formato del diagnóstico inyectado incluye el escenario BDD original, la traducción previa que falló, el error del parser, el análisis completo del explicador (tipo, ubicación, explicación y sugerencia de corrección) y las recomendaciones adicionales. De este modo, el modelo de traducción recibe el contexto completo del fallo y puede aplicar la corrección de manera informada. El prompt de reintento añade además una lista de verificación final que instruye al modelo a confirmar, antes de emitir la respuesta, que la corrección aborda el error específico identificado, que todos los corchetes están correctamente ubicados, que la sintaxis de gates es válida y que la traducción mantiene la correspondencia semántica con el escenario BDD original.

El procesamiento de reintentos opera de manera individual: cada caso fallido se reenvía como una solicitud independiente a la API, utilizando el mismo mecanismo de Structured Outputs y reintentos con *exponential backoff* del módulo de traducción inicial. Esta independencia permite que el fallo de un reintento no afecte la corrección de los demás casos.

4.5.4. Fusión No Destructiva y Re-validación

Las traducciones corregidas se integran al conjunto de resultados mediante una estrategia de fusión no destructiva. El sistema carga las traducciones originales, construye un mapa indexado por identificador de caso con las traducciones corregidas, y recorre secuencialmente las traducciones originales: para cada caso cuyo identificador coincide con una corrección, se reemplaza la traducción original por la corregida; los casos que no presentaron errores se preservan intactos. Este diseño garantiza que las traducciones exitosas nunca se sobrescriban como efecto colateral del proceso de corrección.

Tras la fusión, el pipeline ejecuta una re-validación sintáctica sobre el archivo de traducciones actualizado, aplicando el mismo analizador Earley

utilizado en la validación inicial. Esta re-validación cumple dos funciones: confirmar que las correcciones resolvieron efectivamente los errores de parsing, e identificar casos donde el reintento no logró producir una traducción conforme. Los resultados de la re-validación se persisten en el mismo archivo de validación, actualizando el estado de cada caso.

Las etapas de análisis de errores y reintento se clasifican como no críticas dentro del pipeline: si alguna de ellas falla—por ejemplo, por un error de la API o un agotamiento de la cuota—el pipeline continúa con las traducciones originales y emite una advertencia. Esta decisión se fundamenta en que es preferible contar con traducciones parcialmente correctas (las exitosas de la primera pasada) que interrumpir todo el proceso por un fallo en la corrección de los casos problemáticos.

Capítulo 5

Evaluación

5.1. Preguntas de Investigación

5.2. Diseño Experimental

5.3. Dataset

5.4. Métricas

5.5. Resultados

5.6. Análisis

Capítulo 6

Discusión

6.1. Interpretación de Resultados

6.2. Limitaciones

6.3. Amenazas a la Validez

Capítulo 7

Conclusiones y Trabajo Futuro

7.1. Trabajo Futuro

Una extensión relevante del sistema es la incorporación de un módulo de **validación semántica** que complemente la validación sintáctica implementada en este trabajo. La validación sintáctica verifica que las traducciones IBDD se ajusten a la gramática formal, pero no evalúa si el contenido de la traducción preserva el significado del escenario BDD original. Una traducción puede ser sintácticamente correcta y al mismo tiempo semánticamente deficiente: por ejemplo, podría omitir una entidad mencionada en el escenario, utilizar un gate de salida donde el BDD describe una acción del usuario, o perder un valor numérico relevante para una postcondición.

Durante el desarrollo de este trabajo se analizó la viabilidad de un validador semántico multi-capa que combinara heurísticas léxicas (cotejo de entidades, acciones y valores numéricos entre el texto BDD y la traducción IBDD), verificación de consistencia numérica, y el uso del propio LLM como juez semántico. Este análisis reveló que la validación semántica constituye un desafío de considerable complejidad: la naturaleza abreviada de IBDD—donde variables como `JF` representan entidades como "job file" y gates como `?submit` condensan frases completas—dificulta el cotejo directo con el lenguaje natural, y la evaluación de la correspondencia semántica entre construcciones lingüísticas y elementos formales requiere un nivel de comprensión que excede las capacidades de las heurísticas convencionales. Se concluye que la validación semántica automatizada de traducciones BDD→IBDD es una línea de investigación abierta que merece un tratamiento dedicado en trabajos posteriores.

Bibliografía

[1] Petra van den Bos y Jan Tretmans. "Coverage-Based Testing with Symbolic Transition Systems". En: *Tests and Proofs (TAP 2019)*. Vol. 11823. Lecture Notes in Computer Science. Springer, 2019, págs. 64-82.

[2] Laura Brandán Briones. "Theories for Model-based Testing: Real-time and Coverage". Tesis doct. Enschede, The Netherlands: University of Twente, 2007.

[3] Cucumber Ltd. *Cucumber Open*. `https://cucumber.io/`. Accessed: 2025. 2024.

[4] Cucumber Ltd. *Gherkin Reference*. `https://cucumber.io/docs/gherkin/reference/`. Accessed: 2025. 2024.

[5] Jay Earley. "An Efficient Context-Free Parsing Algorithm". En: *Communications of the ACM* 13.2 (1970), págs. 94-102.

[6] Jannik Fischbach et al. "SPECMATE: Automated Creation of Test Cases from Acceptance Criteria". En: *2020 IEEE International Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 2020.

[7] Lars Frantzen, Jan Tretmans y René de Vries. "A Symbolic Framework for Model-Based Testing". En: *Formal Approaches to Software Testing and Runtime Verification (FATES/RV 2006)*. Vol. 4262. Lecture Notes in Computer Science. Springer, 2006, págs. 40-54.

[8] Lars Frantzen, Jan Tretmans y Tim A. C. Willemse. "Test Generation Based on Symbolic Specifications". En: *Formal Approaches to Software Testing (FATES 2004)*. Vol. 3395. Lecture Notes in Computer Science. Berlin, Heidelberg: Springer, 2005, págs. 1-15.

[9] goodmami. *python-parsing-benchmarks*. 2026. URL: `https://github.com/goodmami/python-parsing-benchmarks` (visitado 29-01-2026).

[10] goodmami. *python-parsing-benchmarks*. 2026. URL: `https://github.com/goodmami/python-parsing-benchmarks` (visitado 28-01-2026).

[11] Dick Grune y Ceriel J. H. Jacobs. *Parsing Techniques: A Practical Guide*. 2nd. Monographs in Computer Science. Springer, 2008.

[12] Juyong Jiang et al. "A Survey on Large Language Models for Code Generation". En: *ACM Transactions on Software Engineering and Methodology* (2024).

[13] Shanthi Karpurapu et al. "Comprehensive Evaluation and Insights into the Use of Large Language Models in the Automation of Behavior-Driven Development Acceptance Test Formulation". En: *IEEE Transactions on Software Engineering* (2024).

[14] Lark Parser Project. *Lark Documentation*. 2026. URL: `https://lark-parser.readthedocs.io/en/stable/` (visitado 29-01-2026).

[15] Lark Parser Project. *Lark Issue Tracker*. 2026. URL: `https://github.com/lark-parser/lark/issues` (visitado 29-01-2026).

[16] Lark Parser Project. *Lark: a parsing toolkit for Python*. 2026. URL: `https://github.com/lark-parser/lark` (visitado 28-01-2026).

[17] Dan North. *Introducing BDD*. `https://dannorth.net/blog/introducing-bdd/`. Originally published in Better Software magazine, March 2006. 2006.

[18] OpenAI. *GPT-4 Technical Report*. 2023. arXiv: `2303.08774 [cs.CL]`.

[19] OpenAI. *Introducing Structured Outputs in the API*. `https://openai.com/index/introducing-structured-outputs-in-the-api/`. Accessed: 2025. 2024.

[20] Python Packaging Authority. *Lark on PyPI*. 2026. URL: `https://pypi.org/project/lark/` (visitado 29-01-2026).

[21] Python Poetry Team. *poetry-core*. 2026. URL: `https://github.com/python-poetry/poetry-core` (visitado 28-01-2026).

[22] Rico Sennrich, Barry Haddow y Alexandra Birch. "Neural Machine Translation of Rare Words with Subword Units". En: *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*. Berlin, Germany: Association for Computational Linguistics, 2016, págs. 1715-1725.

[23] Mathias Soeken, Robert Wille y Rolf Drechsler. "Assisted Behavior Driven Development Using Natural Language Processing". En: *Objects, Models, Components, Patterns (TOOLS 2012)*. Vol. 7304. Lecture Notes in Computer Science. Berlin, Heidelberg: Springer, 2012, págs. 269-287.

[24] Carlos Solís y Xiaofeng Wang. “A Study of the Characteristics of Behaviour Driven Development”. En: *2011 37th EUROMICRO Conference on Software Engineering and Advanced Applications (SEAA)*. Oulu, Finland: IEEE, 2011, págs. 383-387.

[25] Jan Tretmans. “Model Based Testing with Labelled Transition Systems”. En: *Formal Methods and Testing*. Vol. 4949. Lecture Notes in Computer Science. Heidelberg: Springer, 2008, págs. 1-38.

[26] Mark Utting, Alexander Pretschner y Bruno Legeard. “A Taxonomy of Model-Based Testing Approaches”. En: *Software Testing, Verification and Reliability* 22.5 (2012), págs. 297-312.

[27] Ashish Vaswani et al. “Attention Is All You Need”. En: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 30. 2017.

[28] Vyper Team. *Vyper: Pythonic smart contract language for the EVM*. 2026. URL: `https://github.com/vyperlang/vyper` (visitado 28-01-2026).

[29] Niklaus Wirth. “What Can We Do about the Unnecessary Diversity of Notation for Syntactic Definitions?” En: *Communications of the ACM* 20.11 (1977), págs. 822-823.

[30] Tahmineh Zameni et al. “An Intermediate Language to Integrate Behavior-Driven Development Scenarios and Model-Based Testing”. En: *2024 IEEE International Conference on Software Analysis, Evolution and Reengineering - Companion (SANER-C)*. IEEE, 2024.

[31] Tahmineh Zameni et al. “From BDD Scenarios to Test Case Generation”. En: *2023 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. IEEE, 2023.

[32] Tahmineh Zameni et al. “Sequential Composition of BDD Transition Systems for Model-Based Testing”. En: *Formal Techniques for Distributed Objects, Components, and Systems (FORTE)*. Lecture Notes in Computer Science. Springer, 2025.